Scenario 1

Scenario Name : File Upload
Weak Prompt : Generate test cases for file upload.
What I Feel is Missing: No file types, no size limits, no success/failure criteria, no security angle (malicious file upload)**,single/multiple file upload, drag&drop /only file upload**

 Rewritten Prompt:

Write the test cases for file upload functionality

Instruction: Write all the possible test cases for file upload functionality

Context: When the user clicks the file upload button, a file upload pop-up should be displayed with information about the maximum supported file size, supported file formats, single-file upload restriction, and instructions for using the drag-and-drop option. The user should be able to upload an image either by dragging and dropping it into the designated upload area or by clicking the upload area to open the local file explorer and select an image. The system should support only PNG and JPEG/JPG image formats and allow only one file to be uploaded at a time. Multiple file selection and simultaneous uploads should not be allowed. If the selected image exceeds 100 KB, an appropriate error message should be displayed. Similarly, if the image is smaller than 30 KB, an appropriate error message should be displayed. Only valid PNG or JPEG/JPG images with a file size between 30 KB and 100 KB should be uploaded successfully. If the user attempts to upload an unsupported file format, such as HTML or another potentially malicious file type, the system should immediately reject the file and display an appropriate error message. All file type and size validation rules should apply consistently to both drag-and-drop and browse-file uploads. Whenever a file fails validation, the system should display a clear, user-friendly error message explaining the reason for rejection.

Persona: Act as a senior QA engineer

Output Format: Test Case ID | Scenario | Test Data | Steps | Expected Result |

Tone: Stick with the technical industry standard

Observations:

Criteria	Prompt 1	Prompt 2
Clarity	Very broad and generic	Clear, specific, and detailed
Context	No functional requirements provided	Detailed workflow, validation rules, and expected behavior
File formats	General examples of supported and unsupported formats	Explicitly restricted to PNG, JPG, and JPEG
File size	No specific limits	Minimum 30 KB and maximum 100 KB
Upload methods	General upload functionality	Browse and drag-and-drop
Multiple-file handling	General scenarios	Explicitly prohibits multiple-file selection and simultaneous uploads
Error handling	Generic validation errors	Requires clear, user-friendly rejection messages
Persona	Not specified	Senior QA engineer
Output format	Not specified	Fixed columns: Test Case ID, Scenario, Test Data, Steps, Expected Result

| Expected output | General test coverage | Requirement-driven, structured test cases |

Prompt 1 is effective for exploratory test-case brainstorming, whereas Prompt 2 is more effective for professional QA test design because it supplies the functional requirements and defines the expected deliverable. The second result is the stronger starting point for a formal test suite, subject to requirement traceability, boundary clarification, and test-case review.

Scenario 2 :

Scenario Name : API Testing
Weak Prompt : Write API test cases for user registration.
What I Feel is Missing: No HTTP method, no request/response structure, no status codes to validate, no tool context (Postman/RestAssured). **Email/phone number registration**

 Rewritten Prompt:

Write the test cases for user registration api testing

Instruction: Write all the possible test cases for user registration api testing

Context: When the user registration API is triggered through Postman with a valid email ID, a successful response should be received with the message: "User registered successfully. Please check your email to complete the onboarding process." The API should support only the POST method. If any other HTTP method is used, an appropriate error response should be returned. For successful registration, the response status code should be validated as 200 OK. Only an email ID should be accepted in the request payload. If any other value, such as a phone number, is provided instead of an email ID, an appropriate error response should be returned. If the entered email ID already exists in the database, an error response should be returned with the message: "User is already registered." If the entered email ID is in an invalid format, an appropriate validation error response should be returned.

Persona: Act as a senior QA engineer

Output Format: Test Case ID | Scenario | Test Data | Steps | Expected Result |

Tone: Stick with the technical industry standard

Observations:

Criteria	Prompt 1	Prompt 2
User Request	Write API test cases for user registration.	Write all possible test cases for user registration API testing.
Context	Minimal; only the registration use case is specified.	Explicitly defines API behavior, validation rules, HTTP methods, and expected messages.
Requirements	Generic and open-ended.	Specific and measurable.
Expected Success Status	Assumed `201 Created`.	Explicitly requires `200 OK`.
Success Message	Assumed a generic success response.	Provides the exact expected success message.
Request Payload	Assumes name, email, password, confirm password, and phone.	Specifies that only an email ID is accepted.

Duplicate Email Handling	Generic duplicate-email error.	Defines the exact message: `User is already registered.`
HTTP Method Validation	Not initially defined.	Explicitly states that only POST is supported.
Test Coverage	Covers positive, negative, security, database, and performance scenarios.	Provides extensive coverage of functional, validation, method, payload, response, database, security, and performance scenarios.
Output Format	General tables and explanations.	Specifies the columns: Test Case ID, Scenario, Test Data, Steps, Expected Result.
Alignment with Requirements	Based on assumptions due to limited context.	More closely aligned with the supplied requirements.
Overall Result	Useful for generic test-case brainstorming.	More comprehensive, structured, and suitable for requirement-based API testing.

Scenario 3 :

Scenario Name : Role-Based Access Control
Weak Prompt : Write test cases for user roles
What I Feel is Missing: No role types (Admin, Manager, Viewer), no permission matrix, no unauthorized access scenarios, no app domain.

 Rewritten Prompt:

Write the test cases for Role-Based Access Control

Instruction: [STRICT]Write all the possible functional test cases for Role-Based Access Control

Context: There are two types of admin roles: **Admin Read-Write** and **Admin Read-Only**. Both types of admins can log in to the portal and view all template requests submitted by users.If an admin has **Read-Write** access, they can review the details of a template request and approve or reject it. If an admin has **Read-Only** access, they can only review the details of the template request and cannot approve or reject it.Whenever an admin's permissions are changed, the system should automatically enforce the appropriate access permissions based on the assigned role.

Example: Test Case ID | Scenario | Test Data | Steps | Expected Result |

Persona: Act as a senior QA engineer

Output Format: Generate test cases in tabular format

Tone: Stick with the technical industry standard

Observations:
Evaluation Criteria	Prompt 1	Prompt 2
Prompt	"Write test cases for user roles."	Detailed RBAC requirements with Admin Read-Write and Admin Read-Only roles.
Clarity	Very broad and ambiguous.	Clear and specific.
Business Context	Not provided.	Explicitly describes template

request workflows. |
Role Definitions	Assumes Admin, Manager, and Standard User roles.	Clearly defines Admin Read-Write and Admin Read-Only roles.
Permission Rules	Generic examples of role permissions.	Explicitly defines view, approve, and reject permissions.
Role Change Behavior	Not clearly specified.	Requires automatic enforcement of updated permissions when roles change.
Test Coverage	Basic functional, security, and negative test cases.	Detailed functional, security, workflow, API, and role-transition test cases.
Output Format	General test cases and a test case template.	Structured table with Test Case ID, Scenario, Test Data, Steps, and Expected Result.
QA Perspective	Suitable for initial brainstorming.	Better suited for structured test design and execution.
Practical Usability	Requires significant refinement before execution.	Can be adapted for a test management tool after validation.
Assumptions	Makes assumptions about roles and permissions not specified in the prompt.	More closely follows the provided requirements, although some implementation details may still be assumed.
Negative Testing	Covers general unauthorized access scenarios.	Includes role-based restrictions, API authorization, privilege escalation, and unauthorized state changes.
Security Testing	Basic security coverage.	More comprehensive server-side authorization and session security coverage.
Role-Based Regression Testing	Limited explicit coverage of permission changes.	Explicitly covers role upgrades, downgrades, session behavior, and permission revocation.
Requirement Traceability	Difficult because detailed requirements are missing.	Easier to map test cases to the stated business rules.
Overall Assessment	Useful for general exploration and brainstorming.	More effective for requirement-driven test case generation and QA planning.

Conclusion: Prompt 2 produces more relevant and comprehensive test cases because it provides clear business context, role definitions, permission rules, and output expectations. Prompt 1 is useful for brainstorming, while Prompt 2 is a stronger foundation for structured QA testing.